A Two-Stage Stochastic Capacitated Facility Location Solver with a Service-Level Chance Constraint and Benders Decomposition: Implementation and Computational Study

Mohammad S. Hajibabaie

Abstract

We present an open, reproducible solver for the two-stage stochastic capacitated facility location problem (CFLP) with a joint service-level chance constraint. Facilities are opened in a first stage; under uncertain demand the recourse stage serves customers and pays a penalty for unmet demand. The model is formulated as a sample-average-approximation (SAA) mixed-integer program and solved either as a monolithic extensive form or by an L-shaped Benders decomposition. Because the recourse is relatively complete, the decomposition uses optimality cuts only; we additionally implement Magnanti–Wong/Papadakos Pareto-optimal cuts. A single cut-separation routine is shared by three interchangeable backends: an iterative loop on HiGHS, a single-tree branch-and-Benders-cut on SCIP through a constraint handler, and a single-tree implementation on Gurobi using lazy-constraint callbacks. We validate correctness against the published optima of the OR-Library `cap` instances and against the SAA monolith, quantify the value of the stochastic solution (VSS) and the expected value of perfect information (EVPI) on instances built from real city data, and report the effect of demand volatility and of the chance-constraint risk level. We discuss the practical scaling limit imposed by a plain-Python cut routine and outline the path to larger instances.

1 Introduction

Facility location under demand uncertainty is a canonical problem in operations research and supply-chain design: a decision maker commits to opening facilities before demand is known, and then serves realized demand at minimum cost [2, 14]. Two features make the problem both practically relevant and computationally demanding. First, the first-stage decisions are binary and the second-stage recourse is a continuous transportation problem, so the deterministic equivalent is a large mixed-integer program. Second, a decision maker often cares not only about expected cost but also about a *service level*: the guarantee that demand is fully met in all but a small fraction of outcomes. The latter is naturally expressed as a chance constraint [9, 12].

This work contributes a clean, validated, and fully reproducible implementation of such a model, together with a computational study of the solution methods. The emphasis is on engineering correctness and transparency rather than on a new algorithm: every method is classical and well sourced, and the value lies in a faithful re-implementation whose results can be checked against published optima and against an independent monolithic solve. Concretely, the contributions are:

1. A typed, configuration-driven solver for the two-stage stochastic CFLP with a joint service-level chance constraint, formulated by SAA (Section 2).

2. An L-shaped Benders decomposition with optimality cuts only, optional Pareto-optimal cuts, and three interchangeable backends behind one interface that share a single cut-separation routine (Section 3).

3. Stochastic-quality reporting—VSS, EVPI, and an SAA optimality-gap estimate (Section 4).

4. A reproducible experimental study on real city data and OR-Library instances, with an explicit account of the implementation's scaling limit (Sections 5–6).

2 Problem formulation

Let I index candidate facilities, J customers, and $S = \{1, \dots, N\}$ a finite set of demand scenarios with probabilities p_s. The first stage chooses $y_i \in \{0, 1\}$, equal to one when facility i is opened, at fixed cost f_i. Facility i has capacity s_i. In scenario s, customer j has demand d_{js}, the per-unit transport cost from i to j is c_{ij}, the recourse ships $x_{ijs} \geq 0$ units, and any shortfall $u_{js} \geq 0$ is penalized at q_j per unit. The SAA deterministic equivalent is

$$\min_{y,x,u,z} \quad \sum_{i\in I} f_i y_i + \sum_{s\in S} p_s \left(\sum_{i\in I}\sum_{j\in J} c_{ij} x_{ijs} + \sum_{j\in J} q_j u_{js} \right) \tag{1}$$

$$\text{s.t.} \quad \sum_{i\in I} x_{ijs} + u_{js} = d_{js} \qquad \forall j \in J,\ s \in S, \tag{2}$$

$$\sum_{j\in J} x_{ijs} \leq s_i y_i \qquad \forall i \in I,\ s \in S, \tag{3}$$

$$x_{ijs} \leq d_{js}\, y_i \qquad \forall i \in I,\ j \in J,\ s \in S, \tag{4}$$

$$u_{js} \leq d_{js}\, z_s \qquad \forall j \in J,\ s \in S, \tag{5}$$

$$\sum_{s\in S} p_s z_s \leq \gamma, \tag{6}$$

$$y_i, z_s \in \{0, 1\},\ x_{ijs}, u_{js} \geq 0.$$

The penalty variable u provides *relatively complete recourse*: for any first stage, every scenario subproblem is feasible (in the worst case nothing is served and the penalty is paid).

Strong (disaggregated) link. Constraint (4) is redundant given (3), since $\sum_i x_{ijs} \leq d_{js}$ already follows from (2). It nonetheless tightens the linear-programming (LP) relaxation substantially and is the standard strong CFLP formulation. In our implementation it is also required for the HiGHS presolver to return the published optimum on the validation instances: on the weak aggregated model the presolver performs a reduction that returns a sub-optimal incumbent reported as optimal.

Chance constraint. The binary z_s in (5)–(6) marks a scenario permitted to violate full service. When $z_s = 0$ the big-M link forces $u_{js} = 0$, so scenario s is fully served; (6) caps the probability mass of violating scenarios at the risk level γ [9, 12]. For equal-weight scenarios this is the count form $\sum_s z_s \leq \lfloor \gamma N \rfloor$; with unequal weights from scenario reduction the probability-weighted form is the correct generalization. The big-M coefficient d_{js} is tight because $u_{js} \leq d_{js}$ always holds.

Remark 1 (γ is not ϵ)**.** *The risk level γ in* (6) *is an* in-sample *quantity. The* true *target is ϵ, the probability with which the open facilities fail to fully serve a random demand realization. An SAA solution is conservative for the true constraint only when $\gamma \leq \epsilon$; $\gamma = 0$ is a guaranteed inner approximation. The two are kept as distinct parameters.*

3 Solution methods

3.1 Extensive form and decomposition

Problem (1)–(6) can be solved directly as the monolithic extensive form, which we use as an independent oracle. For decomposition, the Benders master retains the first-stage variables y and z and an epigraph variable θ_s per scenario (multi-cut), minimizing $\sum_i f_i y_i + \sum_s p_s \theta_s$ subject to the service budget (6) and accumulated optimality cuts. For a fixed first stage $(\hat{y}, \hat{z})$ the recourse separates into one LP per scenario,

$$Q_s(\hat{y}, \hat{z}_s) = \min \Big\{ \textstyle\sum_{ij} c_{ij} x_{ij} + \sum_j q_j u_j : (2)\text{–}(5) \text{ at } (\hat{y}, \hat{z}_s) \Big\}. \tag{7}$$

Let π be the (free) duals of the demand rows (2), $\alpha \le 0$ those of the capacity rows (3), and $\delta \le 0$ those of the unmet-link rows (5). Because the dual feasible region does not depend on (y, z), a single dual solution yields a cut valid for *all* (y, z):

$$\theta_s \;\ge\; \sum_j \pi_j d_{js} + \sum_i (\alpha_i s_i)\, y_i + \Big(\sum_j \delta_j d_{js} \Big) z_s. \tag{8}$$

This is an optimality cut in the sense of Van Slyke and Wets [15]; relatively complete recourse means feasibility cuts are never required. To keep every $z_s = 0$ subproblem feasible—full service is feasible only when open capacity covers the scenario demand—the master also carries the valid inequality $\sum_i s_i y_i \ge (\sum_j d_{js})(1 - z_s)$. Because the serve graph is complete, total capacity at least equal to total demand suffices for a feasible transportation, so this inequality is sufficient.

3.2 Pareto-optimal cuts

Facility-location recourse is dual-degenerate: the subproblem dual has multiple optima, and an arbitrary one yields a weak cut. Magnanti and Wong [10] select, among the optimal duals, the one that is strongest at an interior core point. We use the independent-point variant of Papadakos [13], which obtains a Pareto-optimal cut by evaluating the subproblem directly at a core point y^0 (avoiding the optimality-tie equality of the original method). The core point starts at the all-fractional vector $\frac{1}{2}\mathbf{1}$ and is moved halfway toward the current master solution each iteration, so it remains interior.

3.3 Backends

All three backends consume the same cut (8); they differ only in search strategy.

- **Classic (HiGHS, MIT).** An iterative L-shaped loop: solve the master mixed-integer program for a lower bound and a trial $(\hat{y}, \hat{z})$, solve the N scenario LPs for the true cost (an upper bound) and fresh cuts, add the violated cuts, and repeat until the bound gap closes.
- **Branch-and-Benders-cut (SCIP ≥ 8, Apache-2.0).** A single search tree with a PySCIPOpt constraint handler that separates cut (8) on integer-feasible candidates and validates incumbents. HiGHS cannot add lazy constraints, so SCIP is the only open single-tree option. SCIP's dual reductions assume a complete constraint set; with lazily added cuts they can fix first-stage variables and remove the optimum, so they are disabled.
- **Lazy-callback (Gurobi, commercial).** A single tree in which cut (8) is injected from the integer-solution callback. Both single-tree backends are warm-started with cuts evaluated at several first-stage points so the root relaxation is informative.

The solver, modeling layer (Pyomo [6]), and cut routine are decoupled, so adding a backend does not touch the model or the separation logic.

4 Stochastic-quality measures

To assess whether the stochastic model is worthwhile and whether the sample is adequate, we report the standard measures [2, 8, 11]. Let RP be the recourse-problem optimum, WS the wait-and-see value (the probability-weighted average of the per-scenario perfect-foresight optima), and EEV the expected result of the expected-value solution (the cost of the mean-demand first stage evaluated over the scenarios). Then

$$\mathrm{EVPI} = \mathrm{RP} - \mathrm{WS} \geq 0, \qquad \mathrm{VSS} = \mathrm{EEV} - \mathrm{RP} \geq 0, \tag{9}$$

where EVPI is the value of perfect information and VSS the value of the stochastic solution over the deterministic mean-value heuristic. We also estimate the SAA optimality gap of a candidate first stage by a statistical lower bound (the mean of several independent SAA replications) and an upper bound (the candidate evaluated on a large reference sample), following Kleywegt et al. [8], Mak et al. [11].

5 Data and implementation

Instances. Geographic instances are built from the GeoNames `cities5000` dataset [4]: city coordinates and population are real, with population taken as nominal demand (rescaled to a fixed mean for numerical conditioning) and great-circle distance as the transport cost. Capacity and fixed cost are model-supplied by documented rules. Deterministic correctness is checked against the OR-Library `cap` instances and their published optima [1]. Demand scenarios are mean-preserving multiplicative lognormal shocks, reduced from a large Monte-Carlo sample to a working set by k-means.

Software. The model is expressed in Pyomo [6]; the open solvers are HiGHS [7] and SCIP [3], with Gurobi [5] as the commercial backend. The recourse subproblem is solved through the HiGHS Python interface, building the LP once and changing only its right-hand side across solves. Each run records the seed, resolved package and solver versions, and the git commit; raw third-party data is not redistributed but fetched by download scripts that verify SHA-256 checksums for the OR-Library files. The implementation is typed and tested, and the full quality gate (linting, type checking, and the test suite) runs in continuous integration on open solvers only.

6 Computational results

All results use seed 20231015 and are produced by a single script that writes a machine-readable results file and the figures referenced here. Instances are denoted n_f/n_s (facilities/scenarios).

6.1 Validation

Table 1 reports the deterministic CFLP objective against the OR-Library published optima for three instances; the solver reproduces each published value to within rounding. On the stochastic instances, the three Benders backends and the SAA monolith agree on the objective to solver tolerance, which we take as evidence that the decomposition and its cuts are implemented correctly.

Table 1: Deterministic CFLP objective versus OR-Library published optima.

Instance	Solver objective	Published optimum	Relative error
cap71	932615.75	932615.75	0
cap101	796648.44	796648.44	$<10^{-9}$
cap131	793439.56	793439.56	$<10^{-9}$

Table 2: Backend comparison on a common instance (identical objective).

Backend	Iterations/rounds	Cuts	Time (s)
Classic (HiGHS)	75	888	86.5
Branch-and-cut (SCIP)	81	924	4.5
Lazy callback (Gurobi)	104	1155	1.6

6.2 Pareto-optimal cuts

On a deliberately degenerate symmetric ring instance, Pareto-optimal cuts reduce the iteration and cut counts of the classic loop while reaching the identical optimum: the standard cuts require 40 iterations and 195 cuts, whereas the Pareto cuts require 27 iterations and 127 cuts. We note that this advantage is instance-dependent: on non-degenerate geographic instances the standard cuts can converge in fewer iterations, so Pareto cuts are exposed as an option rather than the default.

6.3 Backend comparison and scaling

Table 2 compares the three backends on a common instance; all return the same objective. The single-tree backends are substantially faster than the iterative loop, whose cost is dominated by repeatedly re-solving the master and every scenario subproblem. Table 3 reports Gurobi on a sequence of increasing sizes under a time limit: instances up to roughly twenty facilities are solved to proven optimality, and larger instances return a feasible solution with a reported optimality gap.

6.4 Value of the stochastic solution

Figure 1 shows EVPI and VSS as functions of the demand volatility σ on a geographic instance: both grow with volatility, confirming that the stochastic model and the value of information matter more as demand becomes less predictable. Table 4 reports the effect of the chance-constraint risk level γ. Moving from $\gamma = 0$ (full service in every scenario) to $\gamma = 0.1$ lets the model leave the most demanding scenario partly unserved, which lowers total cost at the price of positive expected unmet demand—the explicit trade-off between cost and service level. Increasing γ further has no effect here: a single permitted violation already suffices, so the service budget is no longer binding.

Figure 2 shows the open facilities for a representative solution, and Figure 3 the Benders lower and upper bounds across iterations of the classic loop.

7 Discussion and limitations

The implementation prioritizes correctness, clarity, and solver independence. Its main limitation is scale. The cut-separation routine is plain Python and solves one recourse LP per scenario at every candidate first stage; the number of candidates grows quickly with the number of facilities,

Table 3: Gurobi scaling under a 120 s time limit.

Size n_f/n_s	Objective	Gap	Status
12/8	234268	0%	optimal
20/15	317439	0%	optimal
30/20	428749	9.7%	time limit
50/30	658742	19.2%	time limit

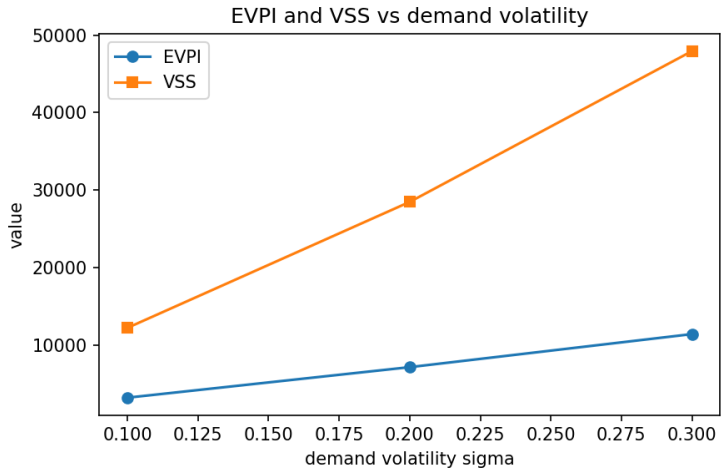

Figure 1: EVPI and VSS versus demand volatility σ.

and the per-candidate cost grows with the number of scenarios. As a consequence, instances of roughly twenty facilities are solved to optimality in seconds to a few minutes, and instances of a few dozen facilities return a bounded-gap solution within a time limit, but instances of one hundred or more facilities are beyond what the current routine solves quickly. This limit is a property of the separation implementation, not of the model or of the commercial solver: the Gurobi backend exhibits the same behavior because it invokes the same Python routine at each integer solution.

8 Conclusion and future work

We have described a validated, reproducible solver for the two-stage stochastic CFLP with a service-level chance constraint, solved by SAA and an L-shaped Benders decomposition with Pareto-optimal cuts and three interchangeable backends. The solver reproduces published OR-Library optima and the independent monolithic objective, and it quantifies the value of the stochastic solution and of perfect information on instances built from real data. The clear next step is to remove the separation bottleneck—by batching or vectorizing the scenario subproblems, or by solving them in a compiled routine—which would extend the approach to the hundred-node instances that the formulation and the commercial backend can in principle handle.

Reproducibility. The source, configurations, tests, and the script that produces every number and figure in this report are available in the project repository. Each run records its seed, package and solver versions, and git commit.

Table 4: Effect of the chance-constraint risk level γ.

γ	Objective	Expected unmet demand
0.0	250954	0.00
0.1	238220	0.109
0.2	238220	0.109
0.3	238220	0.109

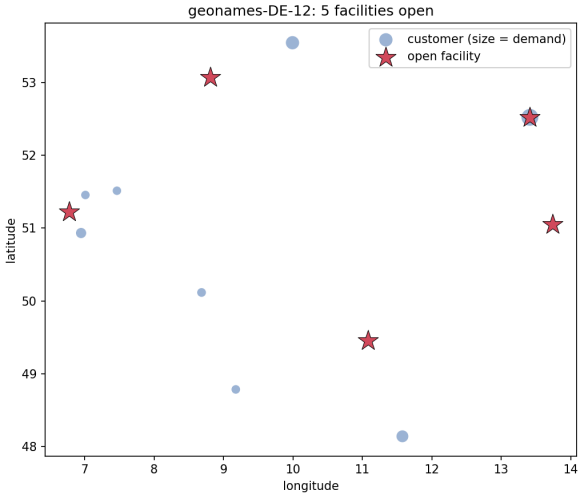

Figure 2: Open facilities (stars) and customers (sized by demand).

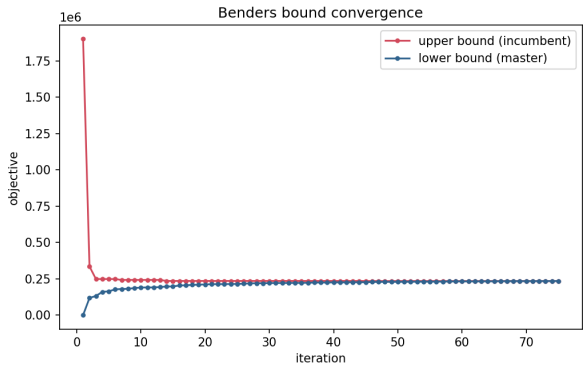

Figure 3: Benders bound convergence (classic loop).

References

[1] J. E. Beasley. OR-Library: distributing test problems by electronic mail. *Journal of the Operational Research Society*, 41(11):1069–1072, 1990. doi: 10.1057/jors.1990.166.

[2] J. R. Birge and F. Louveaux. *Introduction to Stochastic Programming*. Springer, 2nd edition, 2011. doi: 10.1007/978-1-4614-0237-4.

[3] S. Bolusani, M. Besançon, K. Bestuzheva, et al. The SCIP optimization suite 9.0. Technical report, Optimization Online, 2024. URL `https://optimization-online.org/?p=24081`.

[4] GeoNames. GeoNames geographical database, 2024. URL `https://www.geonames.org`. Licensed under CC BY 4.0.

[5] Gurobi Optimization, LLC. Gurobi optimizer reference manual, 2024. URL `https://www.gurobi.com`.

[6] W. E. Hart, J.-P. Watson, D. L. Woodruff, et al. Pyomo: Optimization modeling in python, 2017. Third edition.

[7] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi: 10.1007/s12532-017-0130-5.

[8] A. J. Kleywegt, A. Shapiro, and T. Homem-de Mello. The sample average approximation method for stochastic discrete optimization. *SIAM Journal on Optimization*, 12(2):479–502, 2002. doi: 10.1137/S1052623499363220.

[9] J. Luedtke and S. Ahmed. A sample approximation approach for optimization with probabilistic constraints. *SIAM Journal on Optimization*, 19(2):674–699, 2008. doi: 10.1137/070702928.

[10] T. L. Magnanti and R. T. Wong. Accelerating Benders decomposition: Algorithmic enhancement and model selection criteria. *Operations Research*, 29(3):464–484, 1981. doi: 10.1287/opre.29.3.464.

[11] W.-K. Mak, D. P. Morton, and R. K. Wood. Monte Carlo bounding techniques for determining solution quality in stochastic programs. *Operations Research Letters*, 24(1–2):47–56, 1999. doi: 10.1016/S0167-6377(98)00054-6.

[12] B. K. Pagnoncelli, S. Ahmed, and A. Shapiro. Sample average approximation method for chance constrained programming: theory and applications. *Journal of Optimization Theory and Applications*, 142(2):399–416, 2009. doi: 10.1007/s10957-009-9523-6.

[13] N. Papadakos. Practical enhancements to the Magnanti–Wong method. *Operations Research Letters*, 36(4):444–449, 2008. doi: 10.1016/j.orl.2008.01.005.

[14] T. Santoso, S. Ahmed, M. Goetschalckx, and A. Shapiro. A stochastic programming approach for supply chain network design under uncertainty. *European Journal of Operational Research*, 167(1):96–115, 2005. doi: 10.1016/j.ejor.2004.01.046.

[15] R. M. Van Slyke and R. Wets. L-shaped linear programs with applications to optimal control and stochastic programming. *SIAM Journal on Applied Mathematics*, 17(4):638–663, 1969. doi: 10.1137/0117061.